ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

NOISE MONITOR

Hệ thống giám sát tiếng ồn trong phòng ngủ

Báo cáo tiến độ lần 1 (Checkpoint 1) — Nhóm 14 · PAN

Người thực hiện:

Nguyễn Ngọc Tuấn Anh – 202400029
Nguyễn Hải Yến Nhi – 202400064
Nguyễn Phong – 202400066

Học phần: IoT và ứng dụng (IT4735) · Mã lớp: 175680

Giảng viên: Đặng Tuấn Linh

ONE LOVE. ONE FUTURE.

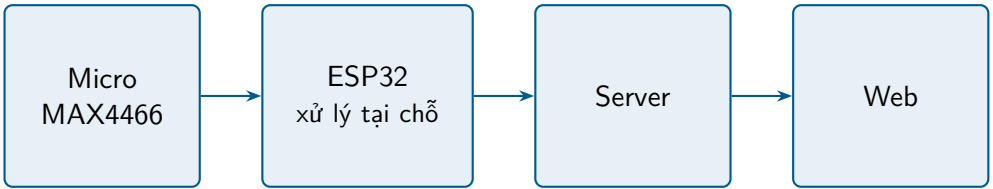
Nội dung trình bày

1. Giới thiệu
2. Yêu cầu hệ thống
3. Thiết kế hệ thống
4. Tiến độ và kế hoạch

1. Giới thiệu

Noise Monitor là gì?

Thiết bị đặt trong phòng ngủ, **đo mức tiếng ồn liên tục cả đêm**, xử lý ngay trên ESP32 và **chỉ gửi số liệu (không ghi âm)** lên server; người dùng xem lại trên web.



chỉ gửi số liệu, không gửi âm thanh thô

Thành viên nhóm

Họ tên	MSSV	GitHub	Phụ trách chính
Nguyễn Ngọc Tuấn Anh	202400029	nnta301	Quy trình repo & CI, backend API, tích hợp hệ thống
Nguyễn Hải Yến Nhi	202400064	nhiyen26	Contract dữ liệu telemetry, dashboard web
Nguyễn Phong	202400066	nguyenphongg233	Firmware ESP32, simulator dữ liệu

Vấn đề và người dùng mục tiêu

Người dùng: sinh viên ở KTX/phòng trọ; người sống gần đường lớn hoặc khu đông dân.

Vấn đề: tiếng ồn ban đêm (xe cộ, hàng xóm, sinh hoạt chung phòng) làm gián đoạn giấc ngủ, nhưng người ngủ không biết ồn lúc mấy giờ, kéo dài bao lâu, lặp lại ra sao; sáng dậy chỉ còn cảm giác "đêm qua ồn".

Cách hiện có & hạn chế:

- App điện thoại ghi âm cả đêm: lộ riêng tư người cùng phòng, tốn pin, phải để máy cạnh giường.
- Máy đo âm thanh cầm tay: chỉ đo tức thời, không lưu lịch sử.

Nhu cầu → giải pháp

Thiết bị rẻ, đặt cố định, đo liên tục, **không ghi âm**, xem lại mức ồn theo thời gian → dữ liệu khách quan để điều chỉnh (đổi vị trí giường, đóng cửa sổ, nút tai, trao đổi với hàng xóm/chủ trọ).

Phạm vi

Đo mức tín hiệu *tương đối* (P2P, đơn vị ADC), không phải dB. Không nhận dạng loại âm thanh. Không đánh giá chất lượng giấc ngủ.

2. Yêu cầu hệ thống

Yêu cầu chức năng

Mã	Yêu cầu	Trạng thái
FR1	Đo âm thanh liên tục, mỗi giây tính mức P2P (trung vị & đỉnh) ngay trên ESP32	Chờ phần cứng code đã xong
FR2	Đóng gói 10 giây dữ liệu, gửi lên server qua Wi-Fi	Đã xong qua simulator
FR3	Server xác thực thiết bị, kiểm tra, lưu trữ, chống trùng dữ liệu	Đã xong
FR4	Web hiển thị mức ồn hiện tại và lịch sử 15 phút	Đã xong
FR5	Báo khi mất dữ liệu / thiết bị ngừng gửi (quá 30 giây)	Đã xong
FR6	Phát hiện ồn kéo dài $\geq$ 30 giây, cảnh báo bằng đèn LED mờ tại thiết bị	Sprint 2
FR7	Ghi nhận sự kiện ồn và báo cáo tổng hợp mỗi đêm	Sprint 2
FR8	Tự kết nối lại và gửi bù dữ liệu khi mất mạng	Sprint 3

Yêu cầu phi chức năng

NFR1 · Riêng tư

Không lưu, không truyền âm thanh thô; chỉ gửi đặc trưng số.

Kiểm chứng: schema chỉ có trường số liệu, gói có trường lạ bị từ chối (400).

NFR2 · Bảo mật

Khóa riêng cho từng thiết bị và cho phía đọc (phân quyền theo thiết bị); không đưa khóa lên repo; HTTPS khi ra Internet.

Kiểm chứng: test 401 (sai khóa), 403 (không có quyền).

NFR3 · Độ trễ

$\geq$ 95% gói hiển thị trên web trong $\leq$ 20 giây (mạng thử nghiệm); giao diện tự cập nhật $\sim$5 giây.

Kiểm chứng: so thời điểm đo với thời điểm hiển thị trong lần chạy thử $\geq$ 30 phút (#7), chưa đo.

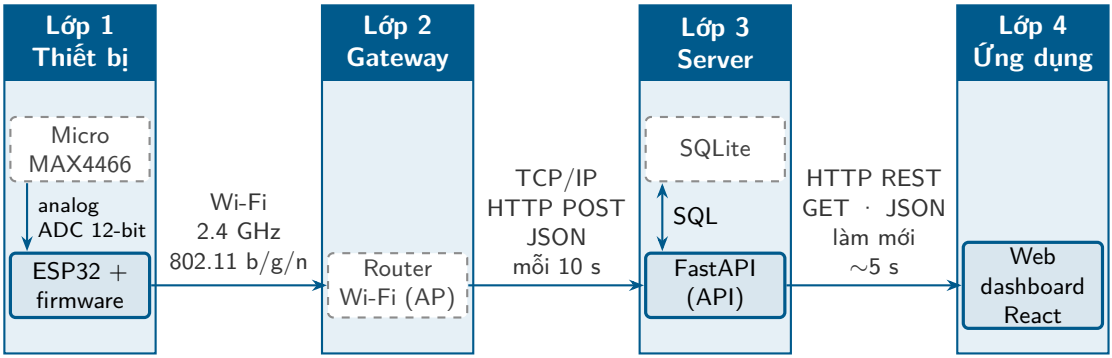
NFR4 · Tin cậy & toàn vẹn

Gửi lại không tạo bản ghi trùng; dữ liệu còn sau khi server khởi động lại; thiếu dữ liệu không bị hiểu là "yên tĩnh".

Kiểm chứng: test chống trùng/409 và test khởi động lại trong backend.

3. Thiết kế hệ thống

Kiến trúc 4 lớp



Không dùng gateway chuyên dụng: ESP32 có Wi-Fi tích hợp, router đóng vai trò gateway. Xử lý tín hiệu tại biên (trên ESP32). Khi triển khai qua Internet dùng HTTPS.

Luồng xử lý dữ liệu

1. ESP32 lấy mẫu ADC (∼500 mẫu/s, mục tiêu best-effort); mỗi cửa sổ 50 ms tính P2P = max − min.
2. Mỗi giây: trung vị & đỉnh của 20 cửa sổ + cờ chất lượng (`ok`/`clipping`/…).
3. Mỗi 10 giây: một gói JSON (device_id, boot_id, sequence, thời điểm đo, 10 giá trị).
4. Server kiểm tra schema → lưu SQLite → trả 202 `accepted`/`duplicate`, hoặc 409 nếu trùng khóa nhưng khác nội dung.
5. Web gọi `/latest` và `/history` để vẽ.

Gói telemetry v1 (rút gọn)

```
{
  "schema_version": "v1",
  "device_id": "esp32-bedroom-01",
  "boot_id": "123e4567-…",
  "sequence": 1,
  "measured_at": "2026-10-19T22:00:00Z",
  "timezone": "Asia/Ho_Chi_Minh",
  "time_sync_status": "synced",
  "median_p2p": [40, 42, 41, …],
  "peak_p2p": [45, 48, 46, …],
  "quality_flags": ["ok", "ok", …]
}
```

Thiết bị và chi phí dự kiến

Linh kiện	Vai trò	SL	Giá tham khảo
ESP32 DevKit V1 (ESP32-WROOM-32, lõi kép)	Vi điều khiển, Wi-Fi	1	∼120.000đ
Module micro MAX4466 (chỉnh được gain)	Cảm biến âm thanh	1	∼45.000đ
LED + điện trở 220 Ω	Báo hiệu ồn kéo dài (Sprint 2)	1	∼5.000đ
Breadboard + dây cắm	Lắp mạch thử	1 bộ	∼40.000đ
Cáp USB + củ sạc 5 V	Cấp nguồn	1	∼70.000đ
Tổng			**∼280.000đ / phòng**

Server chạy trên laptop của nhóm trong giai đoạn thử nghiệm (0đ).

Lợi ích dự kiến: dưới 300.000đ/phòng; không cần để điện thoại cạnh giường; không ghi âm; xem lại được dữ liệu cả đêm.

Giá tham khảo thị trường; nhóm chưa mua linh kiện, sẽ cập nhật theo hóa đơn.

Công nghệ và lý do lựa chọn

Thành phần	Lựa chọn	Lý do chọn	Đã cân nhắc nhưng loại
Vi điều khiển	ESP32	Wi-Fi tích hợp, ADC 12-bit; 2 lõi tách lấy mẫu và gửi mạng	ESP8266: 1 lõi, 1 chân ADC. Arduino Uno: không có Wi-Fi
Firmware	C++ · Arduino · PlatformIO	Build tái lập, chạy được trong CI	Arduino IDE: cấu hình thủ công, khó tái lập và đưa vào CI
Truyền thông	HTTP + JSON (REST)	1 gói/10 s; chống trùng nhờ (boot_id, sequence)	MQTT: phải dựng broker; chưa cần push ở tần suất này
Backend	Python · FastAPI	Kiểm tra dữ liệu bằng JSON Schema; tự sinh tài liệu API	Node.js/Express: phải tự ghép phần kiểm tra dữ liệu và tài liệu API
Cơ sở dữ liệu	SQLite	Một file, không cần máy chủ CSDL; khóa chính chống trùng	PostgreSQL: phải cài, vận hành server; để dành khi mở rộng
Giao diện	React · TS · Recharts	Kiểu dữ liệu chặt; biểu đồ thời gian thực; chạy trên điện thoại	App di động riêng: tốn gấp đôi công sức; web đã chạy trên điện thoại

4. Tiến độ và kế hoạch

Lộ trình 3 sprint (∼2 tuần/sprint)

Sprint 1 (hiện tại)
Pipeline telemetry lõi
Contract · API · dashboard
Simulator · firmware · CI

Sprint 2
Phát hiện ồn & báo cáo đêm
Dữ liệu thật từ ESP32 · NTP
Ngưỡng 30 s + LED · sự kiện
docker-compose

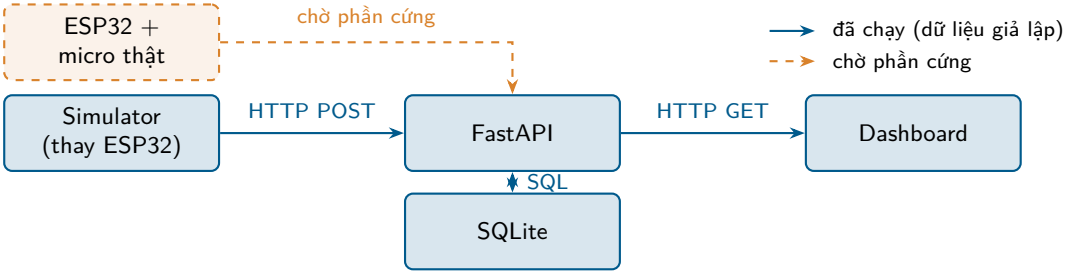
Sprint 3
Độ tin cậy & hoàn thiện
Bộ đệm gửi bù · HTTPS
Test tải & tình huống lỗi
Chạy thử 7 ngày · hồ sơ bàn giao

phần mềm xong, phần cứng chưa

Issue Sprint 1

Đã xong #1 Khởi tạo repo & quy trình
Đã xong #9 CI & hướng dẫn review
Đã xong #2 Contract telemetry v1
Đã xong #4 API nhận & truy vấn dữ liệu
Đã xong #5 Simulator
Đã xong #6 Dashboard
Chờ phần cứng #3 Firmware ESP32: code xong, chưa kiểm chứng trên phần cứng
Đang mở #7 Tích hợp đầu–cuối trên phần cứng

Tiến độ hiện tại: luồng dữ liệu đầu–cuối



- Simulator 5 kịch bản (yên tĩnh, tăng dần, đỉnh ngắn, mất dữ liệu, ngừng gửi) qua API thật: 9 gói được lưu; gửi lại không tạo bản ghi mới; sai khóa → 401; nội dung xung đột → 409.
- Gói JSON do mã C++ của firmware sinh ra được API thật chấp nhận; firmware biên dịch thành công cho ESP32 (2 cấu hình).
- Kiểm thử tự động: 45 test backend, 13 test simulator, 3 test tích hợp; CI chạy trên mọi PR.
- Dashboard: mức mới nhất, biểu đồ 15 phút, trạng thái STALE khi quá 30 giây không có dữ liệu.

Dữ liệu giả lập, chưa có số liệu thật từ micro.

Demo (1/2): nguồn dữ liệu → giao thức → CSDL

❶ Nguồn dữ liệu: simulator thay cảm biến

```
$ python scripts/simulate_telemetry.py --endpoint ... \
    --scenario peaks --interval 1 --duration 90 \
    --start <now-795s>
SYNTHETIC sequence=0 outcome=accepted
...
SYNTHETIC sequence=89 outcome=accepted
```

❷ Giao thức: HTTP + JSON

```
# API=http://127.0.0.1:8000/api/v1
$ curl -X POST $API/telemetry -d @packet_seq89.json \
    -H "X-Device-Key: demo-device-key"
HTTP/1.1 202
{"outcome":"duplicate","packet_key":{...,"sequence":89}}

$ curl -X POST $API/telemetry -H "X-Device-Key: wrong-key" ...
HTTP/1.1 401
{"error":{"code":"device_authentication_failed",...}}

$ curl $API/devices/simulator-synthetic-01/latest \
    -H "X-Read-Key: demo-read-key"
HTTP/1.1 200
{"packet":{"boot_id":"6fb56da2-...","sequence":89,...},
 "freshness":{"status":"fresh","age_seconds":7}}
```

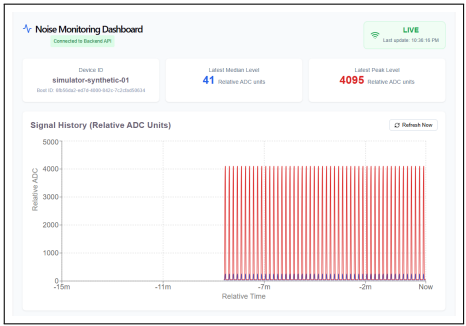
❸ CSDL: bảng packets trong SQLite

```
SELECT COUNT(*) FROM packets
  before re-send: 90
  after  re-send: 90   -> duplicate not stored
boot_id   | seq | measured_at
6fb56da2... |  89 | 2026-11-12T15:36:16Z
6fb56da2... |  88 | 2026-11-12T15:36:06Z
6fb56da2... |  87 | 2026-11-12T15:35:56Z
```

Cùng boot_id 6fb56da2-… ở cả 4 chặng; simulator tua nhanh 10×. **Dữ liệu giả lập.**

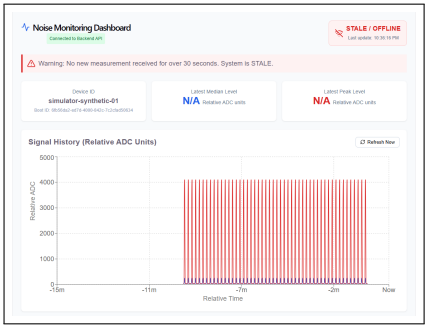
Demo (2/2): giao diện dashboard

4 Đang nhận dữ liệu



LIVE · trung vị 41 · đỉnh 4095 (bão hòa ADC)

4 Ngừng gửi > 30 giây



STALE / OFFLINE · mức hiện tại N/A

Cùng lần chạy và boot_id với trang trước. **Dữ liệu giả lập.**

Khó khăn và bước tiếp theo

Khó khăn

1. Chưa có phần cứng: firmware chưa kiểm chứng thực tế, chưa chọn được gain và ngưỡng.
2. P2P là đơn vị tương đối, chưa hiệu chuẩn sang dB: phải đo thực nghiệm trong phòng để đặt ngưỡng "ồn".
3. ESP32 không có đồng hồ thời gian thực: cần đồng bộ thời gian (NTP) để gán đúng thời điểm đo.
4. Mất mạng: hiện gói lỗi bị bỏ, chưa gửi bù (Sprint 3).

Bước tiếp theo

1. Mua linh kiện, nạp firmware, kiểm tra phản ứng với âm thanh thật.
2. Chạy thử $\geq$ 30 phút trên phần cứng, đo độ trễ (#7).
3. Sprint 2: ngưỡng 30 giây, LED, sự kiện, báo cáo đêm.



Repo: `https://github.com/nnta301/noise-monitor`

Xin cảm ơn thầy và các bạn đã lắng nghe!



THANK YOU !